



Rapport de stage

Conception et développement d'une application Web de gestion des congés pour la fonction publique

Stage réalisé au Centre National de l'Informatique

Stagiaire :	Mohamed Aziz MISSAOUI
Formation :	1ère année d'école d'ingénieurs – Informatique
Établissement :	Grenoble INP – Polytech
Tuteur de stage :	M. Yassine Ben Abdeljelil, Consultant Oracle PL/SQL
Encadrant universitaire :	M. Michaël Périn
Cadre au CNI :	M. Mohamed Ali Saadaoui
Période :	13 juillet – 12 août 2026

Remerciements

Je tiens tout d’abord à exprimer ma sincère gratitude au Centre National de l’Informatique pour m’avoir accueilli dans le cadre de ce stage et pour m’avoir permis de découvrir un environnement professionnel concret.

Je remercie particulièrement mon tuteur de stage, M. Yassine Ben Abdeljelil, Consultant Oracle PL/SQL, pour son accompagnement, ses conseils, sa disponibilité et la confiance qu’il m’a accordée tout au long de cette expérience.

J’adresse également mes remerciements à M. Mohamed Ali Saadaoui, cadre au Centre National de l’Informatique, pour son accueil et son soutien.

Je remercie M. Michaël Périn, responsable du parcours et professeur, ainsi que l’ensemble des enseignants de Grenoble INP – Polytech qui ont contribué à ma formation. Les connaissances et les méthodes acquises m’ont permis de développer les bases techniques nécessaires à ce stage et de renforcer mon autonomie, ma curiosité et surtout ma discipline.

Enfin, je remercie toutes les personnes qui ont contribué, directement ou indirectement, à la réussite de cette expérience.

Résumé

Ce rapport présente la conception et le développement d'un prototype d'application Web de gestion des congés destiné à illustrer le traitement d'une demande dans une administration publique. Le projet répond au besoin de centraliser la consultation des droits, la création des demandes et leur traitement par la hiérarchie.

L'application permet à un agent de consulter son solde, ses jours travaillés et l'historique de ses demandes. Elle intègre un calcul des droits basé sur l'acquisition de 2,5 jours par mois, un **workflow** de validation hiérarchique à plusieurs niveaux, ainsi qu'un historique des décisions. Les administrateurs peuvent gérer les agents, les services, les responsables directs, les soldes et les statuts des agents.

La solution repose sur une architecture **React**, **Spring Boot** et **PostgreSQL**. La sécurité est assurée par **Spring Security**, **JWT**, **BCrypt** et un contrôle des autorisations côté **backend**. Une distribution **Docker** permet de lancer le **frontend**, le **backend** et la base de données sur toute machine équipée de **Docker**.

Mots-clés : gestion des congés, **workflow**, validation hiérarchique, **Spring Boot**, **React**, **JWT**, **PostgreSQL**.

Table des matières

Remerciements	i
Résumé	ii
1 Introduction	1
1.1 Présentation du sujet	1
1.2 Contexte et problématique	1
1.3 Règles métier prises en compte	2
1.3.1 Calcul et utilisation des droits	2
1.3.2 Contrôle des périodes demandées	3
1.3.3 Respect de la hiérarchie	3
1.4 Objectifs du travail	3
1.5 Organisation du rapport	4
2 Contexte et présentation du projet	5
2.1 Structure d'accueil	5
2.2 Besoin identifié	5
2.3 Périmètre du projet	6
2.4 Acteurs et responsabilités	6
2.5 Organisation hiérarchique et workflow	7
2.6 Architecture et technologies retenues	7
2.7 Synthèse du contexte	8
3 Analyse des besoins	9
3.1 Introduction de l'analyse	9
3.2 Besoins fonctionnels	9
3.2.1 Authentification et accès aux espaces	9
3.2.2 Besoins de l'agent	9
3.2.3 Besoins du responsable	10
3.2.4 Besoins de l'administrateur	10
3.3 Besoins non fonctionnels	11
3.3.1 Sécurité	11
3.3.2 Cohérence et intégrité des données	11
3.3.3 Traçabilité	11
3.3.4 Simplicité et lisibilité	11
3.3.5 Disponibilité et portabilité	12
3.4 Règles métier	12
3.4.1 Acquisition et solde	12
3.4.2 Dates et chevauchements	12
3.4.3 Validation hiérarchique	13
3.5 Statuts et cycle de vie d'une demande	13
3.6 Contrôle des erreurs et réponses HTTP	13
3.7 Synthèse des besoins	14
4 Conception de la solution	15

4.1	Introduction	15
4.2	Architecture générale	15
4.3	Organisation des couches du backend	15
4.4	Modèle relationnel	16
4.5	Relations entre les entités	17
4.6	Conception du workflow hiérarchique	17
4.6.1	Contrôle du validateur attendu	18
4.7	Conception de l'authentification et des autorisations	18
4.8	Conception de l'interface	18
4.9	Architecture de déploiement	19
4.10	Synthèse de la conception	19
5	Implémentation	20
5.1	Introduction	20
5.2	Organisation du projet	20
5.3	Implémentation du backend	20
5.3.1	Contrôleurs et routes principales	21
5.3.2	Calcul des droits et gestion des demandes	21
5.4	Authentification et autorisation	21
5.5	Réalisation de l'espace agent	22
5.6	Réalisation de l'espace responsable	24
5.7	Réalisation de l'espace administrateur	26
5.8	Messages, statuts et responsive	29
5.9	Déploiement avec Docker	29
5.10	Synthèse de l'implémentation	30
6	Tests et validation	31
6.1	Introduction	31
6.2	Tests automatisés du backend	31
6.2.1	Tests du calcul des droits	31
6.2.2	Tests du service des demandes	31
6.3	Résultats des tests automatisés	32
6.4	Scénario de validation hiérarchique	32
6.5	Vérification de la consommation du solde	33
6.6	Vérification du refus et des commentaires	34
6.7	Vérification des erreurs métier	34
6.8	Tests de sécurité et d'autorisation	35
6.9	Vérification du déploiement Docker	35
6.10	Bilan de la validation	36
7	Limites et perspectives	37
7.1	Introduction	37
7.2	Limites du prototype actuel	37
7.2.1	Calcul en jours calendaires	37
7.2.2	Types de congés limités	37
7.2.3	Absence de notifications	38
7.2.4	Calendrier non intégré	38
7.2.5	Périmètre administratif et technique	38

7.2.6	Fonctionnalités non couvertes	38
7.3	Perspectives à court terme	38
7.3.1	Notifications internes et électroniques	39
7.3.2	Amélioration de l'initialisation et de la configuration	39
7.3.3	Renforcement des tests	39
7.4	Perspectives à moyen terme	39
7.4.1	Calendrier des congés	39
7.4.2	Gestion des jours ouvrés et des jours fériés	40
7.4.3	Types de congés et règles associées	40
7.4.4	Exports et statistiques	40
7.5	Perspectives à long terme	40
7.5.1	Intégration avec un système de ressources humaines	40
7.5.2	Application mobile et accès distant	41
7.5.3	Configuration avancée des règles	41
7.6	Bilan des perspectives	41
8	Conclusion	42
A	Comptes et scénarios de démonstration	44

1. Introduction

1.1 Présentation du sujet

Dans une administration, la gestion des congés ne se limite pas à enregistrer une date de départ et une date de retour. Elle nécessite de connaître les droits acquis par chaque agent, de vérifier la période demandée, d'identifier les responsables concernés et de conserver une trace des décisions prises. Lorsque ces opérations sont effectuées manuellement ou à l'aide de fichiers séparés, le suivi devient plus difficile et les risques d'erreur augmentent.

Le présent travail porte sur la **conception et le développement d'une application Web de gestion des congés pour la fonction publique**. L'application a été conçue comme une solution centralisée permettant à l'agent de consulter sa situation, de déposer une demande en ligne et de suivre son traitement. Elle fournit également aux responsables les éléments nécessaires à la prise de décision et à l'administrateur les outils de gestion de la structure organisationnelle.

Le sujet comporte trois dimensions complémentaires :

- la conception d'une base de données relationnelle capable de représenter les agents, leurs services, leur statut administratif, leurs responsables et leurs droits à congé ;
- la mise en œuvre des règles de calcul et de contrôle des droits, notamment l'acquisition de 2,5 jours par mois commencé, plafonnée à 30 jours par année ;
- le développement d'un **workflow** de validation hiérarchique permettant de transmettre une demande aux responsables successifs jusqu'à sa décision finale.

La fonctionnalité principale proposée à l'agent est un tableau de bord regroupant son solde de congé restant, les jours acquis et consommés, les informations relatives à son ancienneté ainsi que ses demandes déjà enregistrées. L'agent peut ainsi effectuer une demande en ligne sans transmettre séparément les mêmes informations à plusieurs interlocuteurs.

1.2 Contexte et problématique

La gestion d'une demande de congé implique plusieurs acteurs. L'agent est à l'origine de la demande, mais il ne peut pas toujours être le seul à décider de son acceptation. Le responsable direct doit d'abord vérifier la compatibilité de l'absence avec l'organisation du service. Selon la

position de l'agent dans la hiérarchie, la demande peut ensuite être transmise à un responsable supérieur. La décision doit donc respecter à la fois les droits de l'agent et la chaîne hiérarchique de l'administration.

Un traitement insuffisamment structuré peut provoquer plusieurs difficultés : un solde mal calculé, une période déjà demandée, une validation par une personne non autorisée, une consommation multiple des mêmes jours ou une absence de traçabilité des décisions. La problématique du projet est donc la suivante :

Objectif

Comment concevoir une application Web simple et sécurisée qui permette de gérer les droits à congé d'un agent, de transmettre sa demande selon la hiérarchie et de garantir l'application cohérente des règles métier à chaque étape du processus ?

La validation hiérarchique constitue le cœur fonctionnel du projet. Lorsqu'une demande est créée, elle est affectée au responsable direct de l'agent et reçoit le statut `EN_ATTENTE`. Le responsable concerné est le seul à pouvoir la traiter. S'il l'approuve et qu'un responsable supérieur est défini, la demande reste en attente et est transmise au niveau suivant. Elle ne devient `VALIDEE` qu'après la dernière approbation. À ce moment seulement, les jours sont définitivement déduits du solde. En cas de refus, le processus s'arrête immédiatement et aucun jour n'est consommé.

Cette organisation distingue ainsi trois notions importantes : la demande enregistrée, la demande en cours de validation et la demande définitivement validée. Elle permet également de conserver l'historique de chaque décision, avec le responsable, le niveau de validation, la décision, le commentaire éventuel et la date de traitement.

1.3 Règles métier prises en compte

Le fonctionnement de l'application repose sur des règles métier définies pour le périmètre du projet. Elles sont appliquées côté `backend` afin de ne pas dépendre uniquement des contrôles effectués par l'interface Web.

1.3.1 Calcul et utilisation des droits

Pour le MVP, l'acquisition est fixée à 2,5 jours pour chaque mois commencé. Le total est plafonné à 30 jours par année. Les jours travaillés et la durée d'un congé sont calculés en jours calendaires ; une période du 10 au 12 comprend donc trois jours selon la formule :

$$\text{nombre de jours} = \text{date de fin} - \text{date de début} + 1.$$

Le système tient compte de la date d'embauche pour déterminer les mois acquis. Il prend également en compte les jours déjà consommés et les éventuels ajustements administratifs du solde. Une demande en attente réserve les jours correspondants afin que plusieurs demandes simultanées ne dépassent pas le solde disponible. La consommation définitive est réalisée une

seule fois lors de la validation finale.

1.3.2 Contrôle des périodes demandées

Une demande doit comporter des dates valides : la date de fin ne peut pas être antérieure à la date de début et la date de début ne peut pas se trouver dans le passé. Dans le périmètre actuel, les demandes sont limitées à l'année courante ; les années futures ne sont pas encore ouvertes à la saisie. Le système refuse également une période qui chevauche une demande déjà en attente ou validée pour le même agent. Une demande refusée ou annulée ne bloque pas durablement le solde et ne provoque aucune consommation de jours.

1.3.3 Respect de la hiérarchie

Chaque agent peut être rattaché à un responsable direct. Un responsable peut lui-même dépendre d'un responsable supérieur, ce qui permet de représenter plusieurs niveaux de validation. Lors d'une action sensible, l'identité de l'utilisateur est récupérée à partir de son jeton d'authentification JWT et non d'un identifiant librement transmis par l'interface. Le **backend** vérifie ensuite que l'utilisateur connecté est bien le validateur attendu de la demande.

Cette vérification empêche notamment un agent d'approuver sa propre demande, un responsable de traiter une demande qui ne lui est pas affectée ou un utilisateur de consulter les données d'un autre agent sans autorisation. Les principales erreurs sont signalées par des réponses HTTP adaptées : erreur de saisie ou de règle métier (400), authentification absente ou invalide (401), action interdite (403), ressource inexistante (404) et conflit métier (409).

1.4 Objectifs du travail

L'objectif général est de réaliser une application Web fonctionnelle, cohérente et sécurisée couvrant le cycle principal d'une demande de congé. Les objectifs retenus sont les suivants :

- modéliser une base de données relationnelle représentant les utilisateurs, les agents, les services, les responsables, les soldes, les demandes et les validations ;
- permettre à un agent de consulter son tableau de bord, son solde restant et l'historique de ses demandes ;
- permettre la création en ligne d'une demande avec contrôle des dates, du chevauchement et du solde disponible ;
- mettre en place une validation hiérarchique à un ou plusieurs niveaux ;
- garantir qu'une validation finale consomme les jours une seule fois et qu'un refus ou une annulation ne les consomme pas ;
- fournir à l'administrateur des fonctions de gestion des agents, des services, des responsables, des statuts et des ajustements de solde ;

- sécuriser les accès par authentification **JWT**, hachage **BCrypt** des mots de passe et contrôle des rôles côté **backend** ;
- vérifier le comportement de l'application par des tests unitaires, des tests d'intégration et des scénarios HTTP.

Le projet est volontairement limité à un **MVP**. Il traite les jours comme des jours calendaires et ne gère pas encore automatiquement les jours fériés, les notifications par courriel ou les exports. Ces choix permettent de concentrer le travail sur le processus essentiel : calculer les droits, déposer une demande, la transmettre aux bons responsables et assurer la traçabilité de la décision.

1.5 Organisation du rapport

Le deuxième chapitre présente le contexte d'accueil, les acteurs et le périmètre fonctionnel du projet. Le troisième chapitre analyse les besoins fonctionnels, les contraintes de sécurité et les règles métier. Le quatrième chapitre décrit la conception de l'architecture, de la base de données et du **workflow** hiérarchique. Le cinquième chapitre présente l'implémentation du **backend**, du **frontend** et du déploiement conteneurisé. Le sixième chapitre expose les tests réalisés et les résultats obtenus. Enfin, le septième chapitre présente les limites du **MVP** et les évolutions possibles, avant la conclusion générale.

2. Contexte et présentation du projet

2.1 Structure d'accueil

Le stage a été réalisé au Centre National de l'Informatique, ci-après désigné par son sigle CNI. Le choix d'un sujet consacré à la gestion des congés s'inscrit dans un contexte administratif où les informations relatives aux agents, à leurs droits et à leurs absences doivent être organisées de manière fiable et facilement consultable.

Le projet constitue une réalisation informatique développée dans le cadre du stage. Il ne s'agit pas de reproduire l'ensemble des fonctions d'un système d'information des ressources humaines, mais de concevoir une application ciblée répondant à un processus précis. Cette délimitation permet de traiter de façon approfondie le cycle d'une demande de congé, depuis sa création par l'agent jusqu'à sa décision par la hiérarchie.

2.2 Besoin identifié

Le besoin étudié concerne la centralisation de plusieurs opérations généralement liées entre elles : connaître les droits acquis, vérifier le solde restant, transmettre une demande au bon responsable et conserver les décisions prises. Une application unique facilite la circulation de l'information entre les différents acteurs et réduit les risques liés à la ressaisie ou à l'utilisation de documents séparés.

L'application répond à ce besoin autour de trois fonctions principales :

1. permettre à l'agent de consulter son tableau de bord et son solde de congé restant ;
2. permettre la création et le suivi d'une demande en ligne ;
3. organiser la validation de la demande en fonction de la hiérarchie de l'agent.

Le système prend également en compte le statut de l'agent, son service, son responsable direct et son solde annuel. Ces informations sont liées dans une base de données relationnelle afin que les contrôles métier et les autorisations s'appuient sur des données cohérentes.

2.3 Périmètre du projet

Le projet couvre le périmètre fonctionnel d'un MVP de gestion des congés. Le terme MVP désigne ici une version suffisamment complète pour démontrer le processus essentiel, sans chercher à couvrir toutes les fonctions d'un produit destiné à une administration entière.

Le périmètre réalisé comprend :

- l'authentification des utilisateurs et la séparation des espaces selon leur rôle ;
- la consultation du solde, des jours acquis, des jours consommés et des demandes de l'agent ;
- la création d'une demande avec contrôle des dates, du chevauchement et du solde disponible ;
- l'annulation d'une demande encore en attente et dont la période n'a pas commencé ;
- la validation ou le refus par le responsable attendu ;
- le passage automatique au responsable supérieur lorsqu'un niveau supplémentaire est défini ;
- l'enregistrement de l'historique des décisions ;
- la gestion administrative des agents, des services, des responsables, des statuts et des ajustements de solde.

À l'inverse, le périmètre ne comprend pas la gestion automatique des jours fériés, les notifications par courriel, les exports PDF ou Excel, ni une application mobile. Ces éléments constituent des évolutions possibles, mais ils ne sont pas nécessaires pour démontrer le fonctionnement du processus retenu.

2.4 Acteurs et responsabilités

Le fonctionnement repose sur trois rôles applicatifs. Chaque rôle dispose d'un espace et d'actions adaptés à ses responsabilités.

Acteur	Responsabilités dans l'application
Agent	Consulter son tableau de bord, vérifier son solde, créer une demande, suivre son statut, consulter son historique et annuler une demande autorisée.
Responsable	Consulter les demandes qui lui sont affectées, vérifier les informations de la demande, l'approuver ou la refuser, puis consulter l'historique de ses décisions.
Administrateur	Gérer les agents, les services, les responsables directs, les statuts ACTIF ou SUSPENDU et les ajustements de solde.

TABLE 2.1 – Acteurs et responsabilités de l'application

Le rôle ne constitue pas le seul critère d'autorisation. Par exemple, un utilisateur possédant le rôle **RESPONSABLE** ne peut traiter que les demandes dont il est le validateur attendu. Cette distinction entre rôle général et responsabilité réelle sur une demande permet de respecter la chaîne hiérarchique.

2.5 Organisation hiérarchique et workflow

La hiérarchie est représentée par une relation de responsable direct. Un agent peut être rattaché à un responsable, et ce responsable peut lui-même dépendre d'un autre responsable. Cette relation permet de construire un **workflow** de validation à plusieurs niveaux sans imposer un nombre fixe d'étapes.

Lorsqu'un agent crée une demande, celle-ci est affectée à son responsable direct et reçoit le statut **EN_ATTENTE**. Le premier responsable est donc identifié à partir des données de l'agent, et non choisi librement dans le formulaire. Après une approbation intermédiaire, la demande reste **EN_ATTENTE**, mais son prochain validateur devient le responsable supérieur. La dernière approbation entraîne le passage à **VALIDEE** et la consommation du solde. Un refus entraîne le statut **REFUSEE** et met fin au processus sans consommer les jours.

Cette organisation apporte deux garanties. Premièrement, chaque demande est traitée par les personnes prévues dans la hiérarchie. Deuxièmement, l'application peut expliquer l'état d'une demande grâce à son niveau courant et à son historique de validations.

2.6 Architecture et technologies retenues

L'application repose sur une architecture en trois parties :

- un **frontend** développé avec **React**, **TypeScript** et **Vite**, chargé de l'interface et des interactions avec l'utilisateur ;
- un **backend** développé avec **Java 21** et **Spring Boot**, chargé d'exposer l'API, d'appliquer les règles métier et de contrôler les autorisations ;
- une base de données **PostgreSQL**, chargée de conserver les utilisateurs, les agents, les services, les soldes, les demandes et les validations.

Les échanges entre le **frontend** et le **backend** utilisent des requêtes **HTTP** et des données au format **JSON**. L'authentification repose sur un jeton **JWT**, tandis que les mots de passe sont protégés par **BCrypt**. La couche d'accès aux données utilise **Spring Data JPA**, ce qui permet de relier les classes Java aux tables relationnelles.

Le projet peut être exécuté avec **Docker**. Trois conteneurs sont prévus : un conteneur pour **PostgreSQL**, un pour le **backend Spring Boot** et un pour le **frontend React** servi par **Nginx**. Cette organisation permet de lancer l'application avec ses composants principaux sans installer séparément **Java**, **Maven**, **Node.js** ou **PostgreSQL** sur la machine d'accueil. Le lancement s'effectue depuis la racine du projet avec `docker compose up -build`.

2.7 Synthèse du contexte

Le projet répond donc à un besoin précis : fournir un outil centralisé pour gérer les congés dans le respect des droits de l'agent et de la hiérarchie. Le contexte, les acteurs et l'architecture étant définis, le chapitre suivant pourra détailler les besoins fonctionnels, les contraintes non fonctionnelles et les règles métier qui guident la conception de la solution.

3. Analyse des besoins

3.1 Introduction de l'analyse

L'analyse des besoins permet de transformer le sujet du stage en exigences concrètes pour l'application. Elle précise les actions attendues, les informations nécessaires, les contrôles à effectuer et les limites du périmètre retenu. Cette étape sert également à distinguer ce que chaque acteur peut faire de ce qui doit rester interdit.

L'application est construite autour d'un processus principal : un agent consulte ses droits, dépose une demande, puis celle-ci est examinée par les responsables définis dans sa chaîne hiérarchique. L'administration assure la gestion des données nécessaires à ce processus, notamment les agents, les services, les responsables et les soldes.

3.2 Besoins fonctionnels

3.2.1 Authentification et accès aux espaces

L'utilisateur doit pouvoir s'authentifier à l'aide de son adresse électronique et de son mot de passe. Après une connexion réussie, l'application identifie son rôle et lui présente l'espace correspondant. Les rôles retenus sont **AGENT**, **RESPONSABLE** et **ADMIN**.

L'authentification repose sur un jeton **JWT** transmis dans l'en-tête **Authorization** sous la forme **Bearer**. Le jeton permet au **backend** d'identifier l'utilisateur pour chaque requête protégée. L'interface ne doit donc pas permettre à un utilisateur de choisir le rôle ou l'identité utilisés pour une action sensible.

3.2.2 Besoins de l'agent

L'agent constitue le demandeur du congé. Son espace doit lui permettre de :

- consulter ses informations personnelles et sa date d'embauche ;
- consulter les jours acquis, les jours consommés et les jours disponibles pour l'année courante ;
- consulter le nombre de mois acquis et les jours travaillés ;
- créer une demande en indiquant une date de début, une date de fin et un motif éventuel ;

- suivre le statut de chacune de ses demandes ;
- consulter l'historique des validations associées à une demande ;
- annuler une demande encore **EN_ATTENTE** lorsque le congé n'a pas commencé.

La demande est créée pour l'agent authentifié. L'interface peut afficher les informations de l'agent, mais le **backend** doit utiliser l'identité issue du jeton **JWT** afin d'éviter qu'un utilisateur crée ou consulte une demande au nom d'une autre personne.

3.2.3 Besoins du responsable

Le responsable doit pouvoir traiter les demandes qui lui sont actuellement affectées. Son espace doit lui permettre de :

- consulter la liste des demandes à traiter ;
- connaître l'agent demandeur, son service, les dates et le nombre de jours ;
- identifier le niveau de validation attendu ;
- repérer les demandes urgentes, définies dans le projet comme celles dont le début est prévu dans les trois prochains jours ;
- approuver ou refuser une demande ;
- ajouter un commentaire à sa décision ;
- consulter l'historique de ses validations.

Le responsable ne doit pas voir une liste générale lui permettant de traiter n'importe quelle demande. Le **backend** filtre les demandes selon le champ **prochainValideur** et vérifie à nouveau l'identité du responsable au moment de l'approbation ou du refus.

3.2.4 Besoins de l'administrateur

L'administrateur assure la préparation et la maintenance des données nécessaires au fonctionnement de l'application. Il doit pouvoir :

- consulter la liste des agents et des responsables ;
- créer un agent ou un responsable avec son compte utilisateur ;
- rattacher un agent à un service ;
- définir ou modifier son responsable direct ;
- créer et consulter les services ;

- modifier le statut de l'agent entre **ACTIF** et **SUSPENDU** ;
- enregistrer un ajustement du solde annuel dans la limite prévue par l'application.

La gestion de la hiérarchie doit empêcher une relation incohérente. Un agent ne peut pas être son propre responsable et une modification ne doit pas créer de boucle dans la chaîne hiérarchique. Le retrait d'un responsable est également refusé lorsqu'il existe des demandes **EN_ATTENTE** qui dépendent encore de cette relation.

3.3 Besoins non fonctionnels

3.3.1 Sécurité

La sécurité constitue une exigence essentielle, car l'application manipule des informations personnelles et des données relatives aux absences. Les mots de passe sont enregistrés sous forme hachée avec **BCrypt** et ne doivent jamais être renvoyés dans les réponses de l'API. Les accès sont contrôlés côté **backend**, même lorsqu'une restriction existe déjà dans l'interface.

Les autorisations dépendent à la fois du rôle et du contexte de l'action. Un utilisateur doit être authentifié pour accéder aux ressources protégées. Un agent ne peut consulter que ses demandes, un responsable ne peut traiter que les demandes dont il est le validateur attendu et les routes d'administration sont réservées au rôle **ADMIN**.

3.3.2 Cohérence et intégrité des données

Les règles métier doivent être appliquées de façon identique quel que soit le client utilisé pour accéder à l'API. Le **backend** vérifie donc les dates, les chevauchements, le solde disponible, le validateur attendu et l'état courant de la demande.

La base de données doit garantir les relations essentielles : un compte utilisateur est associé à un agent, un agent appartient à un service, un solde correspond à une année pour un agent et une validation appartient à une demande. Un même agent ne doit pas posséder plusieurs soldes pour la même année.

3.3.3 Traçabilité

Chaque décision prise par un responsable doit être conservée. L'historique indique le validateur, le niveau, la décision, le commentaire et la date. Cette traçabilité permet à l'agent et aux responsables autorisés de comprendre le chemin suivi par une demande et de distinguer une approbation intermédiaire d'une validation finale.

3.3.4 Simplicité et lisibilité

L'interface doit présenter les informations utiles sans multiplier les écrans inutiles. Les espaces sont séparés par rôle et les statuts sont affichés de manière identifiable. Les messages de succès

et d'erreur doivent expliquer le résultat de l'action, par exemple lorsqu'une demande est refusée parce que le solde est insuffisant ou qu'une période est déjà utilisée.

3.3.5 Disponibilité et portabilité

L'application doit pouvoir être lancée de manière reproductible sur une machine équipée de **Docker**. Le déploiement regroupe la base **PostgreSQL**, le **backend** et le **frontend** servi par **Nginx**. Les données sont conservées dans un volume Docker afin que l'arrêt et le redémarrage des conteneurs ne suppriment pas automatiquement la base.

3.4 Règles métier

3.4.1 Acquisition et solde

Le modèle retenu pour le **MVP** prévoit une acquisition de 2,5 jours pour chaque mois commencé, avec un plafond annuel de 30 jours. Le calcul tient compte de la date d'embauche lorsque celle-ci intervient en cours d'année.

La durée d'une demande est calculée en jours calendaires selon la formule :

$$\text{nombre de jours} = \text{dateFin} - \text{dateDebut} + 1.$$

Le solde disponible est obtenu en tenant compte des jours acquis, des ajustements éventuels, des jours déjà consommés et des jours réservés par les demandes **EN_ATTENTE**. Une demande ne peut donc pas être créée si la réservation de ses jours fait dépasser le solde disponible.

3.4.2 Dates et chevauchements

Les contrôles suivants sont appliqués à la création d'une demande :

- les deux dates sont obligatoires ;
- la date de fin doit être postérieure ou égale à la date de début ;
- la date de début ne peut pas être antérieure à la date courante ;
- les dates doivent appartenir à l'année courante dans le périmètre actuel ;
- la période ne doit pas chevaucher une demande **EN_ATTENTE** ou **VALIDEE** du même agent ;
- l'agent doit posséder un responsable direct pour lancer le processus.

Une demande **REFUSEE** ou **ANNULEE** ne consomme aucun jour et ne constitue plus une réservation active pour une nouvelle demande.

3.4.3 Validation hiérarchique

À sa création, une demande reçoit le statut `EN_ATTENTE`, le niveau 1 et le responsable direct comme prochain validateur. Si ce responsable approuve et qu'un responsable supérieur existe, la demande reste `EN_ATTENTE`, son niveau est augmenté et son prochain validateur est mis à jour.

Lorsque le responsable qui traite la demande n'a pas de supérieur responsable configuré, son approbation est considérée comme finale. La demande passe alors à `VALIDEE` et les jours sont ajoutés aux jours consommés. Le refus, quel que soit le niveau, fait passer la demande à `REFUSEE`, supprime le prochain validateur et arrête le processus.

Validation

La consommation du solde est volontairement placée au moment de la validation finale. Les approbations intermédiaires enregistrent une décision et transmettent la demande, mais elles ne consomment pas une seconde fois les jours.

3.5 Statuts et cycle de vie d'une demande

Une demande possède l'un des statuts suivants :

Statut	Signification
<code>EN_ATTENTE</code>	La demande est enregistrée et attend la décision du validateur indiqué.
<code>VALIDEE</code>	Toutes les validations nécessaires ont été obtenues et le solde a été consommé.
<code>REFUSEE</code>	Un responsable a refusé la demande ; le processus est terminé sans consommation du solde.
<code>ANNULEE</code>	L'agent a annulé une demande encore en attente avant le début du congé.

TABLE 3.1 – Statuts d'une demande de congé

Le passage d'un statut à un autre dépend de l'action effectuée et de son auteur. Une demande `EN_ATTENTE` peut être approuvée, refusée ou annulée dans les conditions prévues. Une demande `VALIDEE`, `REFUSEE` ou `ANNULEE` ne peut plus être traitée comme une demande en attente.

3.6 Contrôle des erreurs et réponses HTTP

L'API utilise des réponses HTTP explicites afin que l'interface puisse afficher un message adapté et que les tests puissent vérifier chaque règle :

Code	Cas traité
200	Requête exécutée avec succès, par exemple consultation ou validation autorisée.
400	Données invalides ou règle métier non respectée : dates incohérentes, année non autorisée ou solde insuffisant.
401	Authentification absente ou jeton JWT invalide.
403	Action interdite pour le rôle ou pour l'utilisateur courant.
404	Ressource inexistante, comme une demande ou un agent introuvable.
409	Conflit avec l'état des données, comme un chevauchement ou une relation hiérarchique impossible.

TABLE 3.2 – Principales réponses HTTP prévues

3.7 Synthèse des besoins

L'analyse montre que l'application doit satisfaire simultanément trois exigences : fournir un service simple à l'agent, respecter la chaîne de validation et protéger les données côté serveur. Les règles de solde et de dates ne peuvent pas être séparées du **workflow**, puisque la demande doit être contrôlée avant sa création et que la consommation du solde ne doit intervenir qu'à la dernière approbation.

Ces besoins serviront de base au chapitre suivant. La conception y traduira les responsabilités identifiées en composants logiciels, en entités relationnelles et en relations entre les différents niveaux de validation.

4. Conception de la solution

4.1 Introduction

La conception traduit les besoins identifiés au chapitre précédent en une organisation technique cohérente. Elle définit les composants de l'application, les responsabilités de chaque couche, les relations entre les données et le déroulement du **workflow** de validation. L'objectif est de séparer clairement l'interface, les règles métier et la persistance des données, tout en conservant une circulation contrôlée de l'information.

4.2 Architecture générale

L'application suit une architecture en trois niveaux : un **frontend** pour l'interface utilisateur, un **backend** pour l'API et les règles métier, et une base de données relationnelle pour la conservation des informations.

- Le **frontend** est développé avec **React**, **TypeScript** et **Vite**. Il affiche les espaces correspondant aux rôles, collecte les données des formulaires et présente les réponses reçues de l'API.
- Le **backend** est développé avec **Java 21** et **Spring Boot**. Il expose les routes **REST**, authentifie les utilisateurs, vérifie les autorisations et applique les règles liées aux dates, aux soldes et aux validations.
- La base **PostgreSQL** conserve les comptes, les agents, les services, les soldes, les demandes et l'historique des validations.

Les échanges entre le **frontend** et le **backend** sont réalisés avec des requêtes **HTTP**. Les données sont transmises au format **JSON** et les requêtes protégées comportent un jeton **JWT** dans l'en-tête **Authorization**. Cette organisation permet de maintenir une séparation entre ce que l'utilisateur voit et les contrôles réellement appliqués au serveur.

4.3 Organisation des couches du backend

Le **backend** est organisé en couches afin de limiter le couplage entre les responsabilités :

Contrôleurs Les classes du package **controller** reçoivent les requêtes HTTP, extraient les informations d'authentification et appellent les opérations nécessaires. Elles exposent notamment la connexion, les demandes de congé, les validations et l'administration.

Services Les classes du package **service** portent les règles métier. Le service d'acquisition calcule les droits, tandis que le service des demandes contrôle la disponibilité du solde, l'approbation, le refus, l'annulation et l'enregistrement des décisions.

Repositories Les interfaces du package **repositories** assurent l'accès aux données avec **Spring Data JPA**. Elles permettent de rechercher les agents, les soldes, les demandes et les validations selon les critères nécessaires au processus.

Entités Les classes du package **entity** représentent les objets persistés et leurs relations avec les tables de la base **PostgreSQL**.

Cette séparation permet de conserver des contrôleurs lisibles et de tester les règles principales indépendamment de l'interface. Elle évite également de placer les contrôles importants dans le seul **frontend**, où ils pourraient être contournés par un autre client de l'API.

4.4 Modèle relationnel

Le modèle de données est construit autour des entités suivantes :

Entité	Responsabilité
User	Compte de connexion contenant l'adresse électronique, le mot de passe haché et le rôle applicatif.
Agent	Informations administratives de l'agent, statut, service, date d'embauche et responsable direct.
Service	Structure organisationnelle à laquelle un agent est rattaché.
LeaveBalance	Solde annuel d'un agent : jours acquis, consommés et ajustement éventuel.
LeaveRequest	Demande de congé avec ses dates, sa durée, son motif, son statut et son prochain valideur.
Validation	Décision historique prise par un responsable à un niveau donné.

TABLE 4.1 – Principales entités du modèle relationnel

Un objet **User** est associé à un seul **Agent**. Cette séparation permet de distinguer les informations d'authentification des informations administratives. L'agent possède un rôle à travers son compte, mais conserve également son propre statut, qui peut être **ACTIF** ou **SUSPENDU**.

Un **Agent** peut appartenir à un **Service** et être lié à un autre agent par la relation **responsableDirect**. Cette relation réflexive représente la hiérarchie. Un agent peut posséder plusieurs soldes **LeaveBalance**, mais un seul par année grâce à la contrainte d'unicité sur le couple agent-année.

Une **LeaveRequest** appartient à un agent. Elle conserve son prochain valideur et son niveau courant pour permettre au système de retrouver l'étape suivante du **workflow**. Chaque décision

est enregistrée dans une entité **Validation** liée à la demande, ce qui permet de conserver plusieurs validations pour un même congé.

4.5 Relations entre les entités

Les principales relations peuvent être résumées ainsi :

- un **User** correspond à un **Agent** ;
- un **Agent** est rattaché à un **Service** ;
- un **Agent** peut avoir un responsable direct qui est lui-même un **Agent** ;
- un **Agent** possède un solde **LeaveBalance** par année ;
- un **Agent** peut créer plusieurs **LeaveRequest** ;
- une **LeaveRequest** peut posséder plusieurs objets **Validation** ;
- chaque **Validation** identifie le responsable qui a pris la décision.

Cette organisation permet de retrouver le responsable attendu sans le demander dans le formulaire de congé. Elle permet également d'afficher à l'agent son historique et de fournir au responsable l'historique de ses propres décisions.

4.6 Conception du workflow hiérarchique

Le **workflow** est conçu autour d'un validateur courant et d'un niveau de validation. Ces deux informations sont enregistrées dans **LeaveRequest** afin de connaître à tout moment l'étape où se trouve la demande.

Le déroulement est le suivant :

1. l'agent authentifié crée une demande ;
2. le système vérifie les dates, le chevauchement, le solde disponible et l'existence d'un responsable direct ;
3. la demande est créée avec le statut **EN_ATTENTE**, le niveau 1 et le responsable direct comme prochain validateur ;
4. le responsable attendu consulte la demande et choisit de l'approuver ou de la refuser ;
5. en cas d'approbation intermédiaire, une entrée **Validation** est créée, le niveau est augmenté et le responsable supérieur devient le prochain validateur ;
6. en cas d'approbation finale, une entrée **Validation** est créée, le statut passe à **VALIDEE** et

les jours sont consommés ;

7. en cas de refus, une entrée **Validation** est créée, le statut passe à **REFUSEE** et aucun jour n'est consommé ;
8. l'agent peut annuler une demande encore **EN_ATTENTE** si sa date de début n'est pas atteinte.

La consommation du solde est volontairement séparée des approbations intermédiaires. Ainsi, le passage d'un responsable au suivant ne modifie pas les jours consommés. Cela évite de déduire plusieurs fois la même période et garantit que seule la décision finale produit l'effet administratif définitif.

4.6.1 Contrôle du validateur attendu

Le rôle **RESPONSABLE** autorise l'accès général aux opérations de validation, mais il ne suffit pas à autoriser une demande précise. Avant chaque approbation ou refus, le **backend** compare l'identité de l'utilisateur authentifié avec le champ **prochainValideur** de la demande. Une différence provoque une réponse 403.

Cette vérification est importante dans le cas d'une hiérarchie à plusieurs niveaux. Le responsable du niveau suivant peut approuver uniquement après l'approbation du niveau précédent, puisque son identité n'est placée dans **prochainValideur** qu'au moment de la transmission.

4.7 Conception de l'authentification et des autorisations

Lors de la connexion, le **backend** recherche le compte à partir de l'adresse électronique et compare le mot de passe fourni avec sa version hachée par **BCrypt**. Si les informations sont correctes, un jeton **JWT** est généré avec l'identité et le rôle de l'utilisateur.

Pour chaque requête protégée, **Spring Security** vérifie le jeton et reconstruit l'identité de l'utilisateur. Les routes sont ensuite protégées selon le rôle requis. Les routes d'administration exigent **ADMIN**, les routes d'approbation exigent **RESPONSABLE** et les fonctions de consultation ou de demande sont accessibles aux rôles prévus par le besoin fonctionnel.

Le contrôle est réalisé à deux niveaux : la configuration de sécurité filtre l'accès général aux routes, puis les contrôleurs et services vérifient le contexte précis de l'action. Cette double vérification est nécessaire pour distinguer un responsable autorisé à valider une demande d'un responsable qui n'est pas le validateur attendu de cette demande.

4.8 Conception de l'interface

L'interface est organisée en espaces correspondant aux rôles. Une navigation latérale donne accès aux sections principales et masque les fonctionnalités qui ne concernent pas l'utilisateur connecté.

- L'espace agent met en évidence le solde, les demandes en attente, le prochain congé et l'historique des demandes.
- L'espace responsable présente les demandes à traiter, leur niveau, leur urgence éventuelle et les actions d'approbation ou de refus. Une section séparée conserve l'historique de ses validations.
- L'espace administrateur regroupe les statistiques de synthèse, la création des agents, l'ajustement des soldes, la gestion des services et la liste des agents.

Les formulaires sont contrôlés dans le **frontend** pour guider l'utilisateur, mais les mêmes règles sont vérifiées dans le **backend**. Les messages de succès et d'erreur sont affichés après les opérations afin de rendre le résultat compréhensible. La mise en page est responsive pour rester utilisable sur des écrans de tailles différentes.

4.9 Architecture de déploiement

Le déploiement est prévu avec trois services **Docker** :

- **database**, basé sur **PostgreSQL** ;
- **backend**, construit à partir du projet **Spring Boot** et exposé sur le port 8080 ;
- **frontend**, construit avec **Vite**, servi par **Nginx** et accessible sur le port 5173.

Le réseau Docker permet au service **frontend** de transmettre les requêtes `/api` au service **backend** sans exposer directement le nom interne du conteneur au navigateur. Le volume `postgres_data` conserve les données de **PostgreSQL** lors d'un arrêt ou d'une reconstruction des conteneurs.

Cette architecture rend la distribution portable : la machine d'accueil doit principalement disposer de **Docker** et d'un navigateur. Les outils de développement comme **Java**, **Maven**, **Node.js** et **PostgreSQL** restent nécessaires uniquement pour le lancement manuel ou le développement local hors conteneurs.

4.10 Synthèse de la conception

La conception obtenue sépare l'interface, l'API, les règles métier et la base de données. Le modèle relationnel représente les acteurs et leurs relations, tandis que les champs de la demande permettent de suivre le validateur courant et le niveau atteint. Cette organisation répond aux besoins analysés : elle protège les accès, applique le calcul des droits, transmet les demandes dans l'ordre hiérarchique et conserve les décisions.

Le chapitre suivant présentera la réalisation concrète de cette conception dans le **backend**, le **frontend** et l'environnement de déploiement.

5. Implémentation

5.1 Introduction

L'implémentation correspond à la réalisation concrète de la conception présentée au chapitre précédent. Elle comprend le développement du **backend**, la construction de l'interface **frontend**, l'intégration de l'authentification et la préparation du déploiement avec **Docker**. L'objectif n'est pas seulement de produire des écrans, mais de relier chaque action de l'utilisateur aux règles métier et aux contrôles appliqués par l'API.

5.2 Organisation du projet

Le projet est réparti en plusieurs dossiers correspondant aux principales responsabilités :

- le dossier **backend** contient l'application **Spring Boot**, les entités, les contrôleurs, les services, les repositories et les tests ;
- le dossier **frontend** contient l'application **React**, les pages, les composants visuels, les styles et le service de communication avec l'API ;
- le dossier **docs** contient la documentation d'installation et la collection de tests **Postman** ;
- les fichiers **docker-compose.yml**, **install.sh** et **stop.sh** facilitent le lancement et l'arrêt de l'application ;
- le dossier **rapport-stage** contient le rapport et les captures utilisées pour illustrer les fonctionnalités.

Cette organisation sépare le code applicatif, les ressources de démonstration et les documents du stage. Elle permet également de lancer le **frontend** et le **backend** séparément pendant le développement, ou ensemble avec **Docker** pour une installation portable.

5.3 Implémentation du backend

Le **backend** est développé avec **Java 21** et **Spring Boot**. Les contrôleurs exposent les routes de l'API, tandis que les services regroupent les opérations nécessitant une logique métier.

5.3.1 Contrôleurs et routes principales

Les contrôleurs sont regroupés par domaine fonctionnel :

- **AuthController** traite la connexion sur la route `/api/auth/login`;
- **AgentController** fournit les informations de l'agent authentifié et son tableau de bord ;
- **LeaveRequestController** gère la création, la consultation, l'annulation et l'accès à l'historique des demandes ;
- **ValidationController** fournit l'historique des validations effectuées par un responsable ;
- **AdminController** regroupe la gestion des agents, des services, des responsables, des statuts et des soldes.

Les routes de l'API utilisent les méthodes **GET**, **POST** et **PUT** selon l'opération. Les données sont reçues et renvoyées au format **JSON**. Les contrôleurs délèguent les opérations métier au service approprié afin de ne pas concentrer tous les contrôles dans une seule classe.

5.3.2 Calcul des droits et gestion des demandes

Le service **LeaveAccrualService** calcule le nombre de mois acquis à partir de la date d'embauche et de la date de référence. Chaque mois commencé apporte 2,5 jours et le résultat est limité à 30 jours par année. Le service renvoie également le nombre de jours travaillés, calculé en jours calendaires.

Le service **LeaveRequestService** regroupe les règles liées au cycle de vie d'une demande. Lors de sa création, il vérifie le solde disponible en déduisant les jours déjà consommés et les jours réservés par les demandes **EN_ATTENTE**. Lors d'une approbation, il enregistre la décision et détermine s'il existe un responsable supérieur. La consommation du solde intervient uniquement lorsque l'approbation est finale.

5.4 Authentification et autorisation

L'authentification est réalisée avec **Spring Security** et un jeton **JWT**. Lors de la connexion, le mot de passe est comparé à sa version hachée par **BCrypt**. Si la connexion réussit, le **backend** renvoie un jeton contenant l'identité et le rôle de l'utilisateur.

Le **frontend** conserve les informations de session nécessaires et le service **Axios** ajoute automatiquement le jeton dans l'en-tête **Authorization** des requêtes. Le **backend** vérifie ensuite la signature du jeton et applique les règles d'accès prévues pour le rôle.

Les routes d'administration sont réservées au rôle **ADMIN**. Les routes d'approbation exigent le rôle **RESPONSABLE**, mais ce rôle général est complété par la vérification du champ **prochainValideur**. Ainsi, un responsable ne peut traiter que les demandes qui lui sont affectées à l'étape courante.

5.5 Réalisation de l'espace agent

L'espace agent rassemble les informations nécessaires au suivi individuel des congés. Le tableau de bord présente les demandes en attente, le prochain congé, le solde disponible, les jours acquis, les jours consommés, les mois acquis et les jours travaillés.

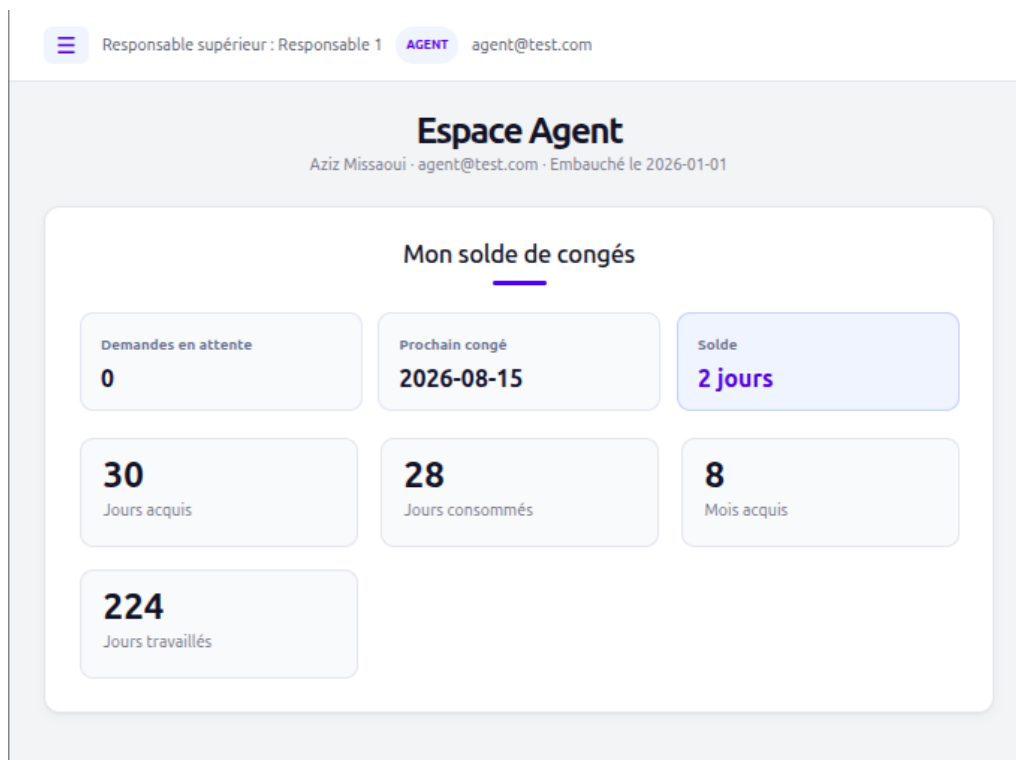
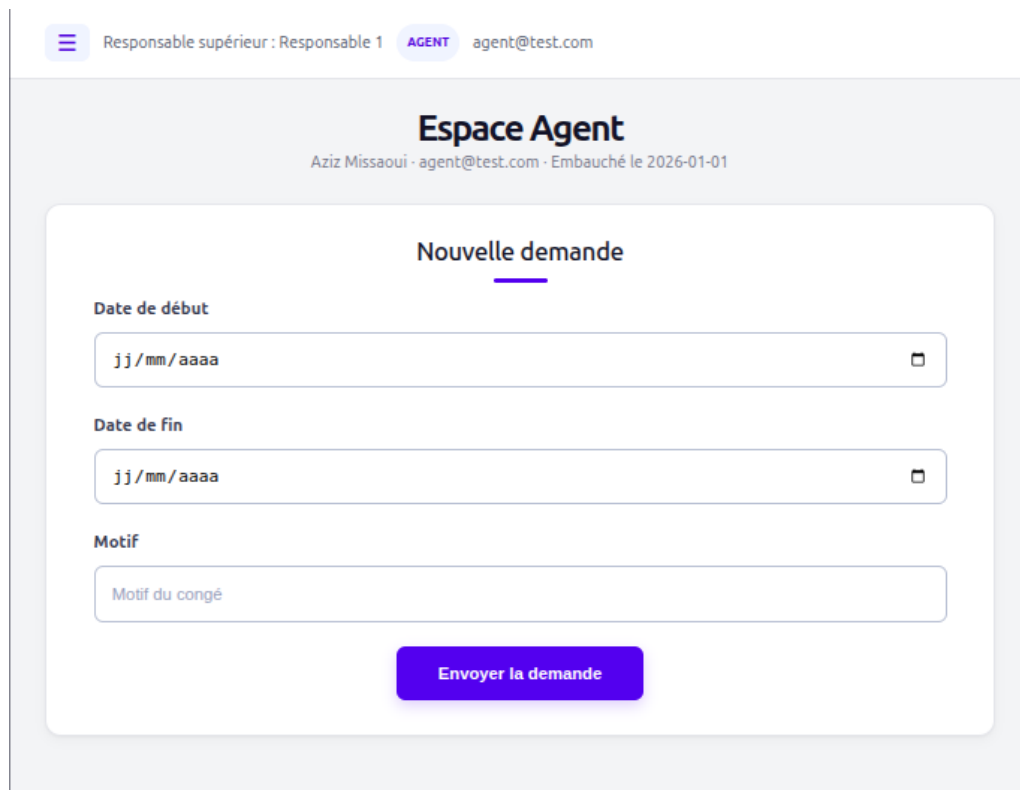


FIGURE 5.1 – Tableau de bord de l'agent

La figure précédente montre l'objectif principal du tableau de bord : donner immédiatement à l'agent une vue synthétique de ses droits. Dans l'exemple, le solde disponible est calculé à partir de 30 jours acquis et de 28 jours consommés. Le nombre de jours travaillés et les mois acquis complètent cette information par des indicateurs liés à la situation de l'agent.

La création d'une demande se fait à partir d'un formulaire comportant une date de début, une date de fin et un motif facultatif. Les contrôles présents dans l'interface facilitent la saisie, mais la validation définitive est effectuée par le **backend**.



The screenshot displays a web application interface for an agent's workspace. At the top, a header bar shows the user's role as 'Responsable supérieur : Responsable 1' and their profile as 'AGENT agent@test.com'. Below this, the main section is titled 'Espace Agent' and includes a subtitle 'Aziz Missaoui - agent@test.com - Embauché le 2026-01-01'. The central focus is a form titled 'Nouvelle demande'. This form contains three input fields: 'Date de début' with a placeholder 'jj/mm/aaaa', 'Date de fin' with a placeholder 'jj/mm/aaaa', and 'Motif' with a placeholder 'Motif du congé'. Each date field has a small calendar icon on the right. At the bottom of the form is a prominent blue button labeled 'Envoyer la demande'.

FIGURE 5.2 – Formulaire de création d'une demande

Après l'enregistrement, l'agent retrouve ses demandes dans une liste. Chaque ligne indique la période, le nombre de jours, le motif, le statut, la progression et l'accès à l'historique. L'action d'annulation est proposée pour les demandes en attente ; le **backend** vérifie ensuite que la période n'a pas commencé.

Mes demandes							
DÉBUT	FIN	JOURS	MOTIF	STATUT	PROGRESSION	HISTORIQUE	ACTION
2026-08-15	2026-08-18	4	Congé personnel	VALIDEE	Terminé	► 0 décision(s)	
2026-08-15	2026-08-18	4	Congé personnel	VALIDEE	Terminé	► 0 décision(s)	
2026-08-15	2026-08-18	4	Congé personnel	ANNULEE	Terminé	► 0 décision(s)	
2026-08-20	2026-08-22	3	Test validation	ANNULEE	Terminé	► 0 décision(s)	
2026-08-15	2026-08-18	4	Congé personnel	VALIDEE	Terminé	► 0 décision(s)	
2026-09-19	2026-09-22	4	mariage	REFUSEE	Terminé	► 0 décision(s)	
2026-08-23	2026-08-27	5	perso	VALIDEE	Terminé	► 0 décision(s)	

FIGURE 5.3 – Liste et suivi des demandes de l’agent

La liste permet notamment de distinguer les demandes **VALIDEE**, **REFUSEE** et **ANNULEE**. L’historique associé à chaque demande peut être ouvert pour consulter les décisions enregistrées. Cette présentation rend visible le résultat du **workflow** sans obliger l’agent à contacter séparément les responsables.

5.6 Réalisation de l’espace responsable

L’espace responsable est conçu autour des demandes qui lui sont assignées. Le tableau de bord indique le nombre de demandes à traiter, le nombre de demandes urgentes et le dernier niveau attendu.



FIGURE 5.4 – Tableau de bord du responsable

Une demande est considérée comme urgente lorsque son début est prévu dans les trois prochains jours. Cette indication aide le responsable à organiser son traitement, sans modifier la règle de décision ni donner une priorité automatique à la demande.

La page des demandes à traiter affiche l’agent, le service, la période, la durée, le motif et le niveau courant. Le responsable peut saisir un commentaire facultatif et choisir entre l’approbation et le refus.

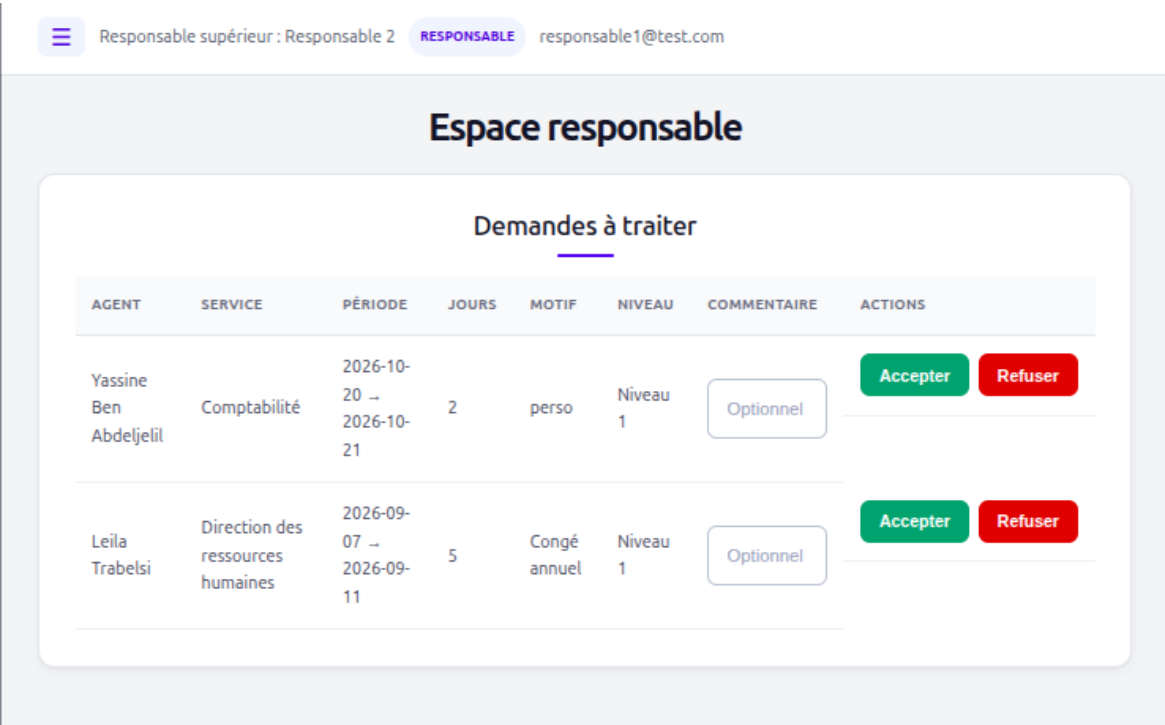


FIGURE 5.5 – Demandes affectées au responsable

Après chaque décision, la demande est retirée de la liste des demandes à traiter pour ce responsable. Si un responsable supérieur existe, elle apparaît ensuite dans son propre espace avec le niveau mis à jour. Les décisions réalisées restent accessibles dans l'historique.

Responsable supérieur : Responsable 2 RESPONSABLE responsable1@test.com

Espace responsable

Historique de mes validations

AGENT	PÉRIODE	JOURS	NIVEAU	DÉCISION	COMMENTAIRE	DATE
Aymen Dridi	2026-08-12 → 2026-08-13	2	Niveau 1	APPROUVEE	-	11/08/2026 22:22:59
Leila Trabelsi	2026-11-02 → 2026-11-04	3	Niveau 1	REFUSEE	Période non compatible avec le service	11/08/2026 21:12:08
Aziz Missaoui	2026-10-16 → 2026-10-20	5	Niveau 1	APPROUVEE	-	11/08/2026 17:25:01

FIGURE 5.6 – Historique des validations du responsable

L'historique présente l'agent concerné, la période, le nombre de jours, le niveau, la décision, le commentaire et la date. Il constitue la traduction visuelle de l'entité **Validation** décrite dans la conception.

5.7 Réalisation de l'espace administrateur

L'espace administrateur permet de suivre la structure gérée par l'application. Le tableau de bord affiche le nombre total d'agents et de responsables, le nombre de responsables et le nombre de services.

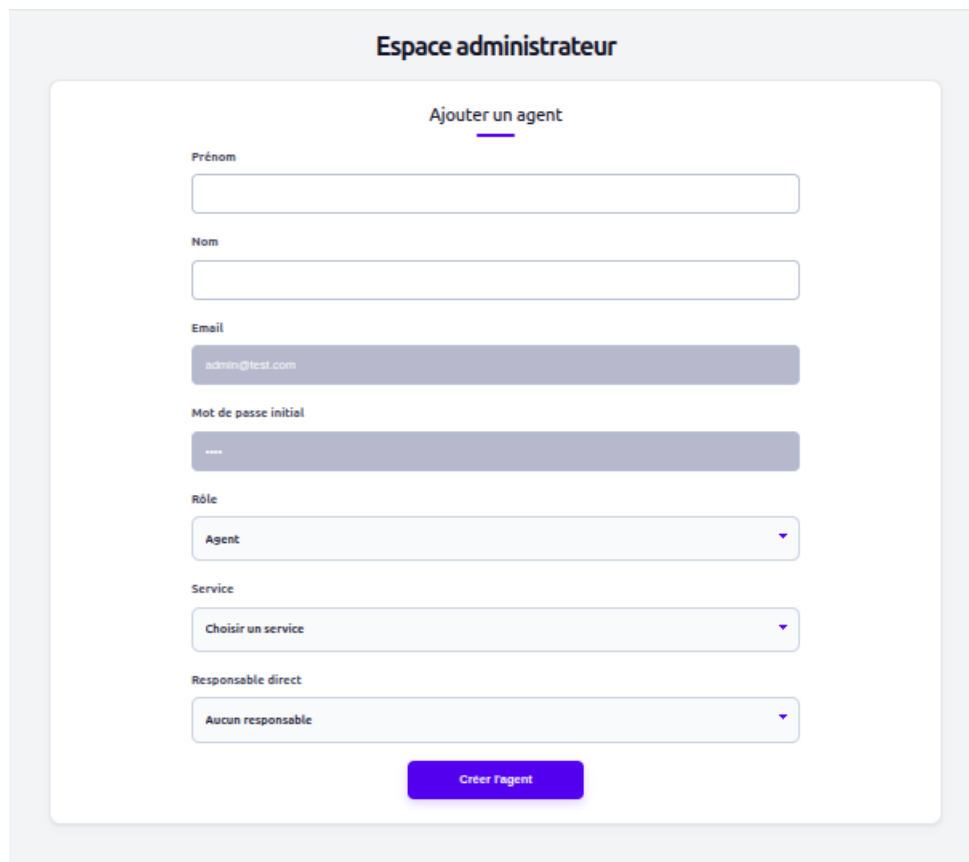
Espace administrateur

11 Agents et responsables	5 Responsables	5 Services
-------------------------------------	--------------------------	----------------------

FIGURE 5.7 – Tableau de bord administrateur

La création d'un agent demande son prénom, son nom, son adresse électronique, un mot de

pas de passe initial, un rôle, un service et un responsable direct éventuel. Le formulaire permet donc de créer le compte et de préparer son rattachement organisationnel au même endroit.



The screenshot shows a web interface titled "Espace administrateur" with a sub-header "Ajouter un agent". The form contains the following fields:

- Prénom**: A text input field.
- Nom**: A text input field.
- Email**: A text input field containing "admin@test.com".
- Mot de passe initial**: A text input field with masked characters (dots).
- Rôle**: A dropdown menu with "Agent" selected.
- Service**: A dropdown menu with "Choisir un service" selected.
- Responsable direct**: A dropdown menu with "Aucun responsable" selected.

At the bottom of the form is a blue button labeled "Créer l'agent".

FIGURE 5.8 – Création d'un agent

La liste des agents donne une vue d'ensemble de la structure. Elle affiche l'adresse électronique, le rôle, le statut, le service, le responsable direct et la date d'embauche. L'administrateur peut modifier le statut entre **ACTIF** et **SUSPENDU** et mettre à jour la relation hiérarchique directement depuis cette liste.

Espace administrateur

Agents

NOM	EMAIL	RÔLE	STATUT	SERVICE	RESPONSABLE DIRECT	DATE D'EMBAUCHE
Responsable 1	responsable1@test.com	RESPONSABLE	Actif	Service Informatique	Responsa	2026-08-11
Aziz Missaoui	agent@test.com	AGENT	Actif	Service Informatique	Responsa	2026-01-01
Leila Trabelsi	demo.leila@cni.tn	AGENT	Actif	Direction des ressources humaines	Responsa	2023-03-15
Sami Mansouri	demo.sami@cni.tn	AGENT	Actif	Direction des ressources humaines	Nadia Ben	2024-06-03
Hatem Jaziri	demo.responsable4@cni.tn	RESPONSABLE	Actif	Direction des ressources humaines	Nadia Ben	2021-09-01

FIGURE 5.9 – Liste et gestion des agents

Les services sont gérés dans une section dédiée. L'administrateur peut créer un service en indiquant son nom et consulter les services existants. Le contrôle d'unicité empêche la création de deux services portant le même nom.

Espace administrateur

Ajouter un service

Nom du service

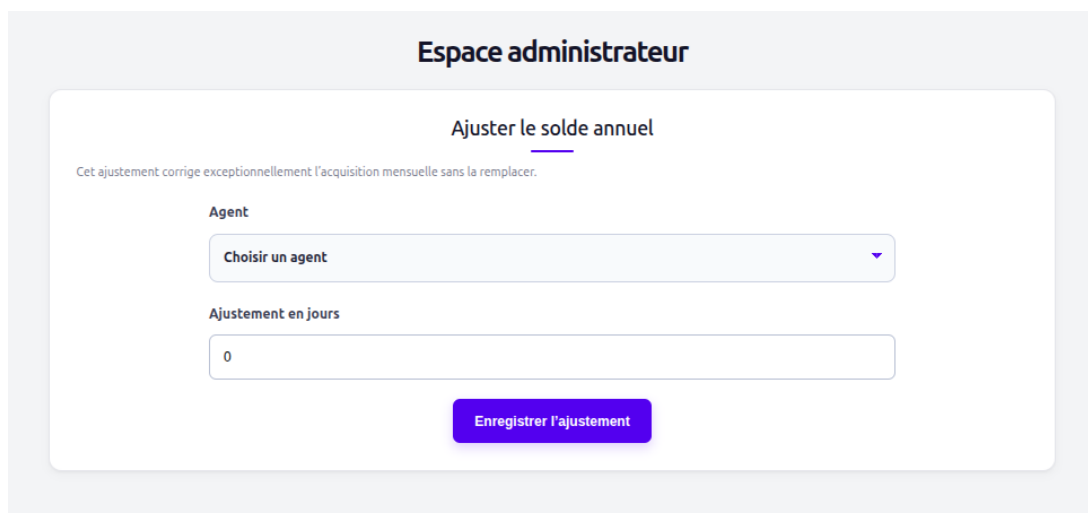
Créer le service

Services existants

Service Informatique
Comptabilité
comptabilité
Administration systèmes et réseaux
Direction des ressources humaines

FIGURE 5.10 – Création et consultation des services

Enfin, l'administrateur dispose d'un formulaire d'ajustement du solde annuel. L'ajustement corrige exceptionnellement l'acquisition calculée sans la remplacer. Il est enregistré séparément afin de conserver la distinction entre les droits calculés automatiquement et la correction administrative.



The screenshot shows a web interface titled "Espace administrateur". Inside, there is a form titled "Ajuster le solde annuel". Below the title, a small note reads: "Cet ajustement corrige exceptionnellement l'acquisition mensuelle sans la remplacer." The form contains two main fields: "Agent" with a dropdown menu labeled "Choisir un agent", and "Ajustement en jours" with a text input field containing the value "0". At the bottom of the form is a blue button labeled "Enregistrer l'ajustement".

FIGURE 5.11 – Ajustement du solde annuel

5.8 Messages, statuts et responsive

Les actions principales affichent un message de succès ou d'erreur après le retour de l'API. Les statuts sont représentés par des éléments visuels distincts afin de faciliter la lecture des listes. Les formulaires utilisent des contrôles adaptés aux dates, aux nombres, aux choix de service et aux rôles.

La navigation latérale est adaptée aux différents espaces. Les fonctionnalités non disponibles pour un rôle ne sont pas proposées dans la navigation. La mise en page utilise des cartes, des tableaux et des indicateurs qui s'adaptent aux écrans de largeur différente.

5.9 Déploiement avec Docker

Le déploiement est défini dans `docker-compose.yml`. Le service `database` utilise l'image `postgres:16-alpine` et conserve les données dans le volume `postgres_data`. Le service `backend` est construit à partir de son `Dockerfile` et attend que la base soit prête grâce à une vérification de santé. Le service `frontend` construit l'application avec `Node.js`, puis la sert avec `Nginx`.

Le fichier `nginx.conf` transmet les requêtes `/api/` au conteneur du `backend` et sert les autres routes comme des ressources du `frontend`. Le script `install.sh` lance la construction et le démarrage avec `docker compose up -build -d`. Le script `stop.sh` arrête les services avec `docker compose down`.

Cette distribution limite les prérequis de la machine d'accueil à `Docker` et à un navigateur. Les ports utilisés sont 5173 pour le `frontend` et 8080 pour le `backend`. Le volume de la base permet de conserver les données lors des redémarrages ordinaires.

5.10 Synthèse de l'implémentation

L'implémentation relie les besoins du projet à trois espaces complémentaires. L'agent dispose d'un tableau de bord et d'un parcours de demande, le responsable dispose des demandes assignées et de l'historique de ses décisions, et l'administrateur dispose des outils de gestion de la structure et des soldes. Les règles importantes restent centralisées dans le **backend**, tandis que le **frontend** fournit une interface lisible et adaptée au rôle de l'utilisateur.

Le chapitre suivant présentera la stratégie de tests utilisée pour vérifier le calcul des droits, la sécurité des accès et le déroulement du **workflow**.

6. Tests et validation

6.1 Introduction

Les tests ont pour objectif de vérifier que l'application respecte les règles métier définies lors de l'analyse et que les fonctionnalités principales restent cohérentes entre l'interface, l'API et la base de données. La validation a été menée à plusieurs niveaux : tests automatisés du **backend**, vérification des scénarios HTTP, tests fonctionnels à partir de l'interface et vérification du déploiement avec **Docker**.

Les scénarios retenus couvrent le parcours principal d'une demande : création par un agent, contrôle du solde, validation par le premier responsable, transmission au niveau supérieur, validation finale ou refus, puis consultation de l'historique et du solde mis à jour.

6.2 Tests automatisés du backend

Les tests automatisés sont exécutés dans le projet **backend** avec **Maven**. Ils vérifient les services métier indépendamment de l'interface graphique.

6.2.1 Tests du calcul des droits

La classe `LeaveAccrualServiceTest` vérifie les règles de calcul des droits :

- acquisition de 2,5 jours pour chaque mois commencé ;
- plafonnement de l'acquisition à 30 jours par année ;
- absence de droits calculés pour une année future non ouverte ;
- calcul des jours travaillés en jours calendaires à partir de la date d'embauche.

Ces tests permettent de vérifier que le calcul ne dépend pas de l'interface et qu'il reste identique lors d'une création de demande ou de l'affichage du tableau de bord.

6.2.2 Tests du service des demandes

La classe `LeaveRequestServiceTest` couvre le comportement du **workflow** et du solde :

- approbation intermédiaire avec maintien du statut `EN_ATTENTE` ;
- transmission au responsable supérieur et augmentation du niveau ;
- validation finale avec consommation du solde ;
- refus d'une demande sans consommation de jours ;
- rejet d'une demande lorsque le solde est insuffisant ;
- réservation des jours par une demande déjà `EN_ATTENTE` ;
- autorisation d'une demande lorsque le solde disponible est suffisant.

Le test d'intégration `GestionCongesApplicationTests` vérifie également que le contexte `Spring Boot` se charge correctement avec le profil de test.

6.3 Résultats des tests automatisés

Les rapports générés par `Surefire` indiquent les résultats suivants :

Classe de test	Tests exécutés	Résultat
<code>LeaveAccrualServiceTest</code>	4	Réussis
<code>LeaveRequestServiceTest</code>	7	Réussis
<code>GestionCongesApplicationTests</code>	1	Réussi
Total	12	Réussis

TABLE 6.1 – Résultats des tests automatisés du backend

Ces résultats confirment les règles principales du modèle : calcul des droits, réservation du solde, progression de la validation et consommation unique lors de l'approbation finale.

6.4 Scénario de validation hiérarchique

Le scénario principal utilise une demande qui traverse deux niveaux de validation. Après sa création par l'agent, la demande est confiée au responsable direct. Celui-ci l'approuve, mais la demande ne devient pas immédiatement validée : elle reste `EN_ATTENTE` et passe au niveau suivant.

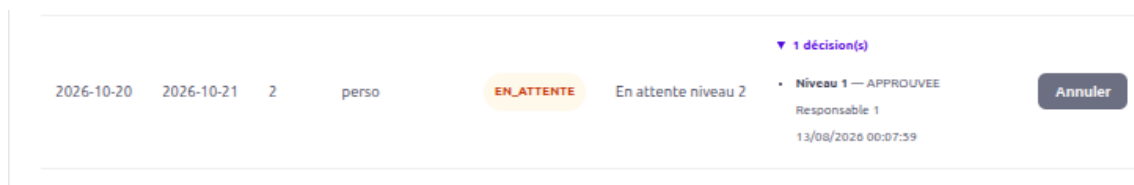


FIGURE 6.1 – Demande en attente après l'approbation du niveau 1

La capture montre une demande toujours en attente, avec la progression indiquant le niveau 2. L'historique indique également la décision **APPROUVEE** du responsable de niveau 1 et la date de traitement. Ce résultat vérifie que l'approbation intermédiaire transmet la demande sans consommer définitivement le solde.

Le responsable du niveau 2 peut ensuite prendre la décision finale. Dans le cas d'une approbation, les deux décisions apparaissent dans l'historique et la demande passe à **VALIDEE**.

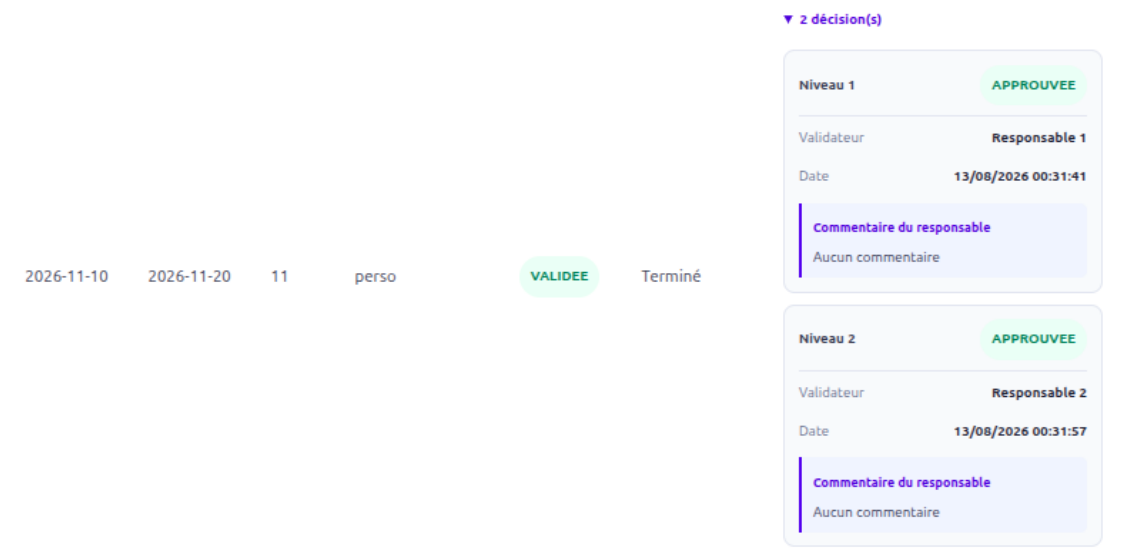


FIGURE 6.2 – Demande validée après les deux niveaux d'approbation

La capture confirme que l'historique conserve les deux décisions, leur niveau, le responsable concerné et la date. Le statut final **VALIDEE** montre que le processus complet est arrivé à son terme.

6.5 Vérification de la consommation du solde

La consommation des jours est vérifiée au moment de la validation finale. Une demande de 11 jours apparaît comme validée dans l'historique, tandis que le tableau de bord indique les jours consommés et le nouveau solde disponible.



FIGURE 6.3 – Solde de l'agent après validation finale

Cette vérification est importante car une approbation intermédiaire ne doit pas modifier les jours consommés. La consommation ne doit apparaître qu'une seule fois, lors de la dernière approbation. Le test automatisé correspondant vérifie également que le solde n'est pas enregistré plusieurs fois pour la même demande.

6.6 Vérification du refus et des commentaires

Le refus termine le **workflow** sans consommer de jours. L'historique associe la décision au responsable qui l'a prise et conserve le commentaire fourni au moment du traitement.

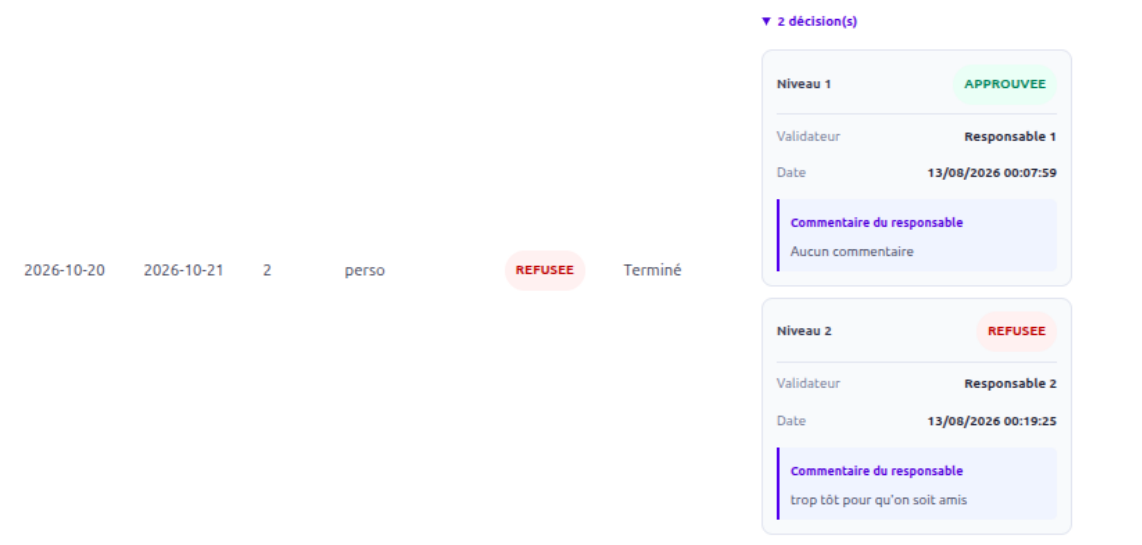


FIGURE 6.4 – Historique d'une demande avec approbation intermédiaire et refus commenté

La capture montre une approbation au niveau 1, suivie d'un refus au niveau 2. Le commentaire du responsable est affiché directement avec la décision correspondante. Cette présentation assure la traçabilité du traitement et permet à l'agent de comprendre la raison du refus.

6.7 Vérification des erreurs métier

Les règles de dates et de chevauchement sont vérifiées avant l'enregistrement d'une demande. Lorsqu'une période est déjà demandée ou validée, le système doit refuser la nouvelle demande avec un conflit métier. L'interface affiche alors un message d'erreur au lieu de créer une nouvelle ligne dans la base.

Nouvelle demande

Date de début

12/12/2026

Date de fin

12/12/2026

Motif

Motif du congé

Envoyer la demande

Impossible de créer la demande.

FIGURE 6.5 – Refus d’une demande en conflit avec une période existante

La même logique est appliquée lorsque le solde disponible est insuffisant, lorsqu’une date est passée ou lorsqu’une demande concerne une année future. Ces contrôles sont effectués côté **backend**, même si l’interface réalise également des contrôles de saisie pour guider l’utilisateur.

6.8 Tests de sécurité et d’autorisation

Les scénarios de sécurité vérifient qu’un utilisateur ne peut pas accéder aux actions qui ne correspondent pas à son identité ou à son rôle :

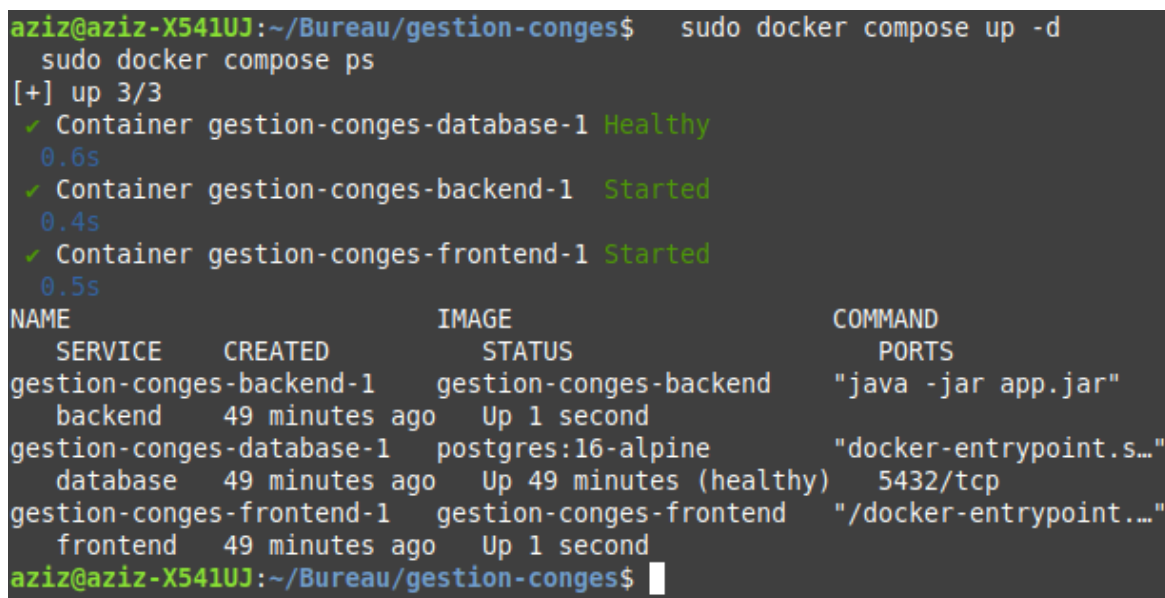
- une requête vers une route protégée sans jeton **JWT** reçoit une réponse **401** ;
- un agent qui tente d’approuver une demande reçoit une réponse **403** ;
- un responsable qui n’est pas le validateur attendu ne peut pas traiter la demande ;
- un agent ne peut consulter que ses propres demandes et leur historique ;
- les routes d’administration sont réservées au rôle **ADMIN**.

Ces contrôles sont appliqués par **Spring Security** et complétés par les vérifications réalisées dans les contrôleurs et les services. L’interface ne constitue donc pas le seul niveau de protection.

6.9 Vérification du déploiement Docker

Le déploiement a été vérifié avec la commande `docker compose up -d`. Les images du **backend** et du **frontend** sont construites à partir de leurs **Dockerfile**, puis les trois services sont

démarrés sur le réseau Docker du projet.



```

aziz@aziz-X541UJ:~/Bureau/gestion-conges$ sudo docker compose up -d
sudo docker compose ps
[+] up 3/3
✓ Container gestion-conges-database-1 Healthy
0.6s
✓ Container gestion-conges-backend-1 Started
0.4s
✓ Container gestion-conges-frontend-1 Started
0.5s

```

NAME	SERVICE	CREATED	IMAGE	STATUS	COMMAND	PORTS
gestion-conges-backend-1	backend	49 minutes ago	gestion-conges-backend	Up 1 second	"java -jar app.jar"	
gestion-conges-database-1	database	49 minutes ago	postgres:16-alpine	Up 49 minutes (healthy)	"docker-entrypoint.s..."	5432/tcp
gestion-conges-frontend-1	frontend	49 minutes ago	gestion-conges-frontend	Up 1 second	"/docker-entrypoint...."	

```

aziz@aziz-X541UJ:~/Bureau/gestion-conges$

```

FIGURE 6.6 – État des services Docker après le démarrage de l'application

La capture confirme que les conteneurs **database**, **backend** et **frontend** sont démarrés. La base est signalée comme **healthy**, ce qui signifie que sa vérification de disponibilité est réussie. L'application est ensuite accessible depuis le navigateur à l'adresse **http://localhost:5173**.

Un conflit de port peut empêcher le démarrage si une autre instance utilise déjà 8080 ou 5173. Ce cas ne correspond pas à une erreur de construction de l'application : il faut arrêter l'instance existante ou libérer le port avant de relancer **docker compose up -d**. Les données du volume **postgres_data** sont conservées avec **docker compose down**, tandis que la commande **docker compose down -v** doit être réservée à une réinitialisation volontaire de la base.

6.10 Bilan de la validation

Les tests réalisés couvrent les éléments essentiels du projet : calcul des droits, contrôle du solde, création d'une demande, validation à plusieurs niveaux, refus, historique, sécurité des accès et déploiement conteneurisé. Les tests automatisés réussissent et les captures fonctionnelles montrent que les résultats attendus sont visibles dans les espaces agent et responsable.

La validation confirme donc que le MVP répond à son objectif principal : permettre à un agent de déposer une demande de congé, la faire traiter par les responsables prévus dans la hiérarchie et conserver une trace claire de chaque décision. Les limites fonctionnelles restantes, comme l'absence de jours fériés ou de notifications par courriel, relèvent du périmètre défini et non d'un échec des tests.

7. Limites et perspectives

7.1 Introduction

Le projet réalisé dans le cadre de ce stage d'un mois est un prototype fonctionnel destiné à démontrer la conception et le développement d'une application Web de gestion des congés. Il ne s'agit pas d'un système destiné à être déployé directement au Centre National de l'Informatique, ni d'un système d'information complet pour la fonction publique.

Le périmètre a été volontairement limité afin de se concentrer sur les éléments essentiels du sujet : la modélisation des données, le calcul des droits, le tableau de bord de l'agent et le **workflow** de validation hiérarchique. Les limites présentées dans ce chapitre correspondent donc principalement à des choix de périmètre liés à la durée du stage et à l'objectif de démonstration.

7.2 Limites du prototype actuel

7.2.1 Calcul en jours calendaires

Le prototype calcule les durées de congé en jours calendaires. Une période allant du 10 au 12 représente donc trois jours, sans distinguer les jours ouvrés, les week-ends ou les jours fériés. Ce choix simplifie le calcul et permet de vérifier clairement le fonctionnement du **workflow**, mais il ne représente pas toutes les règles susceptibles d'être appliquées dans une administration.

Le système ne possède pas encore de calendrier national ou administratif permettant d'exclure automatiquement les jours fériés. Il ne permet pas non plus de configurer des règles différentes selon le service ou la catégorie de l'agent.

7.2.2 Types de congés limités

Le prototype traite principalement une demande de congé générale. Le motif est enregistré comme une information textuelle, mais il ne correspond pas encore à une catégorie métier structurée. Il n'existe donc pas de distinction fonctionnelle entre un congé annuel, un congé exceptionnel, un congé maladie, un congé sans solde ou un congé familial.

Cette simplification permet de conserver un modèle de données simple pour la démonstration, mais elle ne permet pas d'associer des règles spécifiques à chaque type de congé. Par exemple, certains types pourraient avoir un mode de calcul, un justificatif ou une chaîne de validation différente.

7.2.3 Absence de notifications

Le prototype n'envoie pas de notification par courrier électronique ou par notification interne. L'utilisateur doit consulter son espace pour savoir si une demande a été approuvée, refusée ou transmise au niveau suivant.

Cette limite ne remet pas en cause le fonctionnement du **workflow**, car l'état et l'historique de la demande sont bien enregistrés. Elle réduit toutefois le confort d'utilisation dans un contexte réel, où le responsable et l'agent devraient être informés automatiquement des actions importantes.

7.2.4 Calendrier non intégré

Les demandes sont actuellement présentées sous forme de listes et de tableaux. L'utilisateur peut consulter les dates, la durée et le statut, mais il ne dispose pas d'une vue mensuelle ou annuelle permettant de visualiser les absences sur un calendrier.

Une telle vue serait utile pour repérer rapidement les périodes déjà occupées, les congés des membres d'un même service et les périodes comportant plusieurs absences. Son absence constitue donc une limite d'ergonomie et de visualisation, mais elle n'empêche pas le contrôle des chevauchements réalisé par le **backend**.

7.2.5 Périmètre administratif et technique

Les règles retenues sont celles du MVP : acquisition de 2,5 jours par mois commencé, plafond annuel de 30 jours, calcul en jours calendaires et demandes limitées à l'année courante. Le prototype ne gère pas encore les campagnes annuelles, les reports de solde, les changements de règles en cours d'année ou les droits spécifiques à différentes catégories d'agents.

Le déploiement avec **Docker** facilite l'exécution sur une autre machine, mais il nécessite toujours que cette machine dispose de **Docker**. Le projet n'est donc pas un installateur autonome et ne comprend pas encore de publication des images dans un registre ou d'interface d'installation graphique.

7.2.6 Fonctionnalités non couvertes

Le prototype ne comprend pas de récupération de mot de passe par courrier électronique, d'export PDF ou **Excel**, d'application mobile, de statistiques avancées ou d'intégration avec un système externe de ressources humaines. Ces fonctionnalités dépassaient le périmètre d'un stage d'un mois consacré à la réalisation du processus principal.

7.3 Perspectives à court terme

Les améliorations à court terme concernent les fonctions qui pourraient être ajoutées sans modifier profondément l'architecture existante.

7.3.1 Notifications internes et électroniques

Une première évolution serait d'ajouter un système de notifications. Une notification pourrait être créée lorsqu'une demande est déposée, lorsqu'un responsable prend une décision, lorsqu'une demande passe au niveau supérieur ou lorsqu'un congé approche de sa date de début.

Une solution simple consisterait à ajouter une entité **Notification** liée à un utilisateur, avec un message, une date de création et un indicateur de lecture. Une évolution complémentaire pourrait utiliser un service d'envoi de courriels. Les notifications devraient rester complémentaires au tableau de bord : l'historique enregistré dans l'application demeurerait la source de référence des décisions.

7.3.2 Amélioration de l'initialisation et de la configuration

L'installation pourrait être simplifiée par l'automatisation de l'import des données de démonstration et par la création d'un compte administrateur initial contrôlé. La configuration des paramètres sensibles, comme la clé de signature du jeton **JWT**, devrait également être déplacée vers des variables d'environnement plutôt que de rester dans le code source.

Cette évolution améliorerait la reproductibilité du prototype et réduirait les manipulations nécessaires lors d'une nouvelle installation, sans modifier le fonctionnement du **workflow**.

7.3.3 Renforcement des tests

Il serait possible d'ajouter des tests d'API automatisés couvrant directement les réponses **HTTP**, ainsi que des tests de bout en bout simulant les parcours de l'agent et du responsable. Ces tests complèteraient les tests unitaires existants et faciliteraient la détection d'une régression lors d'une modification de l'interface ou des règles métier.

7.4 Perspectives à moyen terme

Les perspectives à moyen terme nécessiteraient une évolution plus importante du modèle de données et de l'interface.

7.4.1 Calendrier des congés

L'ajout d'un calendrier permettrait de représenter les congés dans une vue mensuelle, trimestrielle ou annuelle. L'agent pourrait visualiser ses propres périodes, tandis que le responsable pourrait consulter les absences de son service selon les autorisations prévues.

Le calendrier pourrait utiliser des couleurs différentes pour les demandes **EN_ATTENTE**, **VALIDEE**, **REFUSEE** et **ANNULEE**. Il pourrait également signaler les chevauchements avant même l'envoi du formulaire. Le contrôle réalisé côté **backend** resterait cependant indispensable, car l'affichage du calendrier ne suffit pas à garantir l'intégrité des données.

7.4.2 Gestion des jours ouvrés et des jours fériés

Une évolution du calcul consisterait à introduire une table `Holiday` contenant les jours fériés applicables à une année et éventuellement à une administration. Le service de calcul pourrait alors distinguer les jours calendaires des jours ouvrés selon le type de congé et la règle choisie.

Cette évolution devrait être réalisée avec prudence, car elle modifierait le nombre de jours déduits du solde et les contrôles de disponibilité. Elle nécessiterait également de définir précisément les règles applicables aux périodes qui commencent ou se terminent autour d'un week-end ou d'un jour férié.

7.4.3 Types de congés et règles associées

La gestion de types de congés structurés permettrait de remplacer le simple motif textuel par une catégorie sélectionnée lors de la demande. Une entité `LeaveType` pourrait définir le nom du congé, son mode de déduction, la nécessité d'un justificatif et les responsables autorisés à le traiter.

Le modèle pourrait alors prendre en charge le congé annuel, le congé exceptionnel, le congé maladie, le congé sans solde et les congés liés à des situations familiales. Cette évolution rendrait le système plus proche d'un outil administratif, tout en conservant le principe de validation hiérarchique développé dans le prototype.

7.4.4 Exports et statistiques

Des fonctions d'export permettraient de générer une attestation de congé ou un état des absences au format PDF ou Excel. Des indicateurs pourraient également présenter l'évolution des demandes par mois, la consommation des soldes par service et le nombre de demandes en attente.

Ces informations seraient utiles pour l'analyse, mais elles devraient respecter les autorisations d'accès afin qu'un utilisateur ne puisse pas consulter les données de services qui ne relèvent pas de sa responsabilité.

7.5 Perspectives à long terme

À plus long terme, le prototype pourrait évoluer vers une solution plus complète, capable de s'intégrer dans un environnement administratif plus large.

7.5.1 Intégration avec un système de ressources humaines

Une intégration avec un SIRH permettrait de récupérer les agents, les services, les dates d'embauche et les changements de situation depuis une source officielle. L'application de congés ne serait alors plus responsable de toutes les saisies administratives et pourrait se concentrer sur les demandes et les validations.

Cette intégration nécessiterait de définir des interfaces d'échange, des règles de synchronisation et une gestion des conflits entre les données locales et les données provenant du système externe.

7.5.2 Application mobile et accès distant

Une application mobile pourrait permettre à l'agent de consulter son solde et de répondre rapidement aux demandes de validation. Elle devrait réutiliser l'API sécurisée existante et appliquer les mêmes autorisations que l'application Web.

Un déploiement sur un serveur ou une infrastructure en nuage pourrait également remplacer l'exécution locale par **Docker**. Cette évolution demanderait une configuration plus complète de la sécurité, des sauvegardes, des certificats **HTTPS**, des journaux et de la supervision.

7.5.3 Configuration avancée des règles

Une version plus avancée pourrait permettre à l'administrateur de configurer les règles sans modifier le code : taux d'acquisition, plafond annuel, types de congés, jours fériés, nombre maximal de jours consécutifs et niveaux de validation.

Le **workflow** pourrait également devenir configurable selon le service, le type de congé ou la catégorie de l'agent. Cette flexibilité transformerait le prototype en une plateforme adaptable, mais elle augmenterait fortement la complexité de la conception, des contrôles et des tests.

7.6 Bilan des perspectives

Les limites du projet sont cohérentes avec son objectif : réaliser en un mois un prototype fonctionnel illustrant la gestion des droits, le tableau de bord agent et la validation hiérarchique. Le projet ne prétend pas couvrir l'ensemble des besoins d'une administration ni être directement déployé au **CNI**.

Les perspectives peuvent être hiérarchisées selon leur impact. Les notifications, le calendrier et les types de congés amélioreraient directement l'expérience utilisateur et la richesse fonctionnelle. Les jours fériés, les exports et les statistiques rapprocheraient l'application d'un outil administratif plus complet. Enfin, l'intégration avec un **SIRH**, l'application mobile et la configuration avancée des règles constitueraient des évolutions de long terme.

Aucune difficulté particulière n'a été retenue comme limitation dans le cadre du stage. Les choix présentés ici correspondent donc à des extensions possibles du prototype, et non à des problèmes non résolus lors du développement.

8. Conclusion

Ce stage d'une durée d'un mois a permis de concevoir et de développer un prototype fonctionnel d'application Web de gestion des congés pour la fonction publique. Le travail réalisé répond au sujet fixé : modéliser une base de données relationnelle intégrant les agents, leur statut et leurs droits, mettre en place un **workflow** de validation hiérarchique et fournir à l'agent un tableau de bord avec son solde de congé restant et son formulaire de demande en ligne.

La solution réalisée couvre le cycle principal d'une demande. L'agent peut consulter ses droits, créer une demande et suivre son évolution. Le responsable peut traiter les demandes qui lui sont attribuées selon la chaîne hiérarchique. L'administrateur peut gérer les agents, les services, les responsables directs, les statuts et les ajustements de solde. Les décisions sont enregistrées dans un historique afin de conserver la trace du traitement.

Les règles métier principales ont été intégrées au **backend**. Le système calcule l'acquisition à raison de 2,5 jours par mois commencé, avec un plafond annuel de 30 jours. Il contrôle les dates, les chevauchements et le solde disponible. Les demandes en attente réservent les jours correspondants, tandis que la consommation définitive intervient uniquement lors de la validation finale. Cette organisation garantit qu'une approbation intermédiaire transmet la demande sans consommer plusieurs fois le même solde.

Sur le plan technique, le projet a permis de mettre en pratique une architecture séparant le **frontend**, le **backend** et la base de données. L'interface a été développée avec **React**, **TypeScript** et **Vite**. L'API repose sur **Java 21**, **Spring Boot**, **Spring Data JPA** et **PostgreSQL**. La sécurité utilise **Spring Security**, les jetons **JWT** et le hachage **BCrypt**. Le déploiement avec **Docker** permet de réunir la base, l'API et le site Web dans un environnement reproductible.

Les tests automatisés et les scénarios fonctionnels ont permis de vérifier le calcul des droits, la réservation du solde, la validation à plusieurs niveaux, le refus, l'annulation, les contrôles d'accès et le déploiement conteneurisé. Les captures réalisées dans les espaces agent, responsable et administrateur illustrent le fonctionnement du prototype et rendent visibles les principaux résultats obtenus.

Ce travail a également constitué une expérience de découverte du développement d'une application complète. Il a permis de mobiliser la modélisation relationnelle, la conception d'une API, la sécurisation des accès, l'intégration d'un **frontend** et d'un **backend**, ainsi que la préparation d'un déploiement avec **Docker**. Il a aussi renforcé la compréhension du lien entre les besoins fonctionnels, les règles métier, la conception et les tests.

Le résultat doit être considéré comme un prototype de démonstration réalisé dans le temps

limité du stage, et non comme un système complet destiné à être déployé directement au Centre National de l'Informatique. Les limites conservées dans le périmètre sont notamment l'utilisation de jours calendaires, l'absence de gestion automatique des jours fériés, de notifications, de calendrier intégré et de catégories de congés structurées.

Les perspectives identifiées permettraient d'enrichir progressivement la solution : ajout de notifications, intégration d'un calendrier, gestion de plusieurs types de congés, prise en compte des jours fériés et des jours ouvrés, exports, statistiques, application mobile et éventuelle intégration avec un **SIRH**. Ces évolutions pourraient s'appuyer sur l'architecture et le **workflow** déjà conçus.

En conclusion, ce stage a permis de transformer un besoin administratif en une solution logicielle cohérente et testable. Même limitée à un prototype, l'application montre comment la modélisation des données, les règles de calcul des droits et la validation hiérarchique peuvent être réunies dans un même outil Web. Cette réalisation constitue une première expérience concrète de conception et de développement d'une application complète, depuis l'analyse du besoin jusqu'à sa validation et sa mise en œuvre avec **Docker**.

A. Comptes et scénarios de démonstration

Les comptes de démonstration sont les suivants :

- agent : `demo.leila@cni.tn` / `password`;
- responsable : `demo.responsable3@cni.tn` / `password`;
- administrateur : `admin@test.com` / `test`.

Ces identifiants sont réservés à la démonstration locale et ne doivent pas être utilisés en production.